Experiment 8

Title

Building a Simple Counter App using React Hooks. Use React Hooks (useState, useEffect) to manage state and side effects.

Theory:

React is a popular JavaScript library used for building interactive user interfaces. Traditionally, React applications used **class components** to manage state and lifecycle methods. With the introduction of **React Hooks**, functional components can now manage state and side effects more efficiently.

1. React Hooks

Hooks are special functions in React that allow developers to "hook into" React features such as state and lifecycle methods from functional components. Hooks improve code readability, reusability, and reduce complexity.

2. useState Hook

The useState hook is used to manage **state** in functional components. State refers to data that can change over time and affects what is rendered on the screen.

In a counter application, the counter value is stored as state. Whenever the state changes, React automatically re-renders the component to reflect the updated value.

3. useEffect Hook

The useEffect hook is used to handle **side effects** in functional components. Side effects include:

- Updating the document title
- Fetching data
- Logging values
- Running code after component render

In this experiment, useEffect is used to update the browser title whenever the counter value changes, demonstrating how side effects are handled in React.

4. Component Re-rendering

Whenever the state changes using useState, React re-renders the component. The useEffect hook runs after rendering, ensuring that side effects are synchronized with the component's state.

Algorithm / Procedure

1. Start the experiment.
2. Create a new React application using create-react-app.
3. Open the project folder in VS Code.
4. Create a functional component for the Counter App.
5. Initialize counter state using useState.
6. Add buttons to increment and decrement the counter value.
7. Use useEffect to update the document title when the counter changes.
8. Display the counter value dynamically on the webpage.
9. Run the React application in the browser.
10. Stop the experiment.

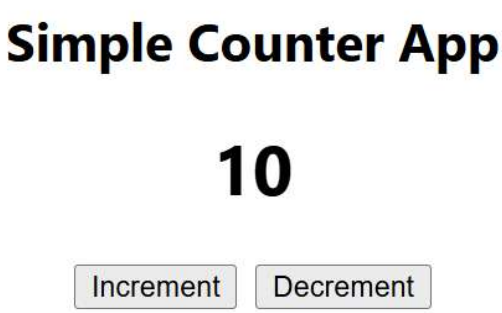
Program / Code:

Counter Component (App.js)

```
import React, { useState, useEffect } from "react";


function App() {
  const [count, setCount] = useState(0);


  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);
```

```
  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h2>Simple Counter App</h2>
      <h1>{count}</h1>

      <button onClick={() => setCount(count + 1)}>Increment</button>
        
      <button onClick={() => setCount(count - 1)}>Decrement</button>
    </div>
  );
}

export default App;
```

Output:



Conclusion

Thus, a simple counter application was successfully developed using React Hooks. The experiment demonstrates the use of useState for state management and useEffect for handling side effects. This approach simplifies component logic and improves code readability in modern React applications.